

Longest palindromic substring

In computer science, the **longest palindromic substring** or **longest symmetric factor** problem is the problem of finding a maximum-length contiguous substring of a given string that is also a palindrome. For example, the longest palindromic substring of "bananas" is "anana". The longest palindromic substring is not guaranteed to be unique; for example, in the string "abracadabra", there is no palindromic substring with length greater than three, but there are two palindromic substrings with length three, namely, "aca" and "ada". In some applications it may be necessary to return all maximal palindromic substrings (that is, all substrings that are themselves palindromes and cannot be extended to larger palindromic substrings) rather than returning only one substring or returning the maximum length of a palindromic substring.

Manacher (1975) invented an $O(n)$ -time algorithm for listing all the palindromes that appear at the start of a given string of length n . However, as observed e.g., by Apostolico, Breslauer & Galil (1995), the same algorithm can also be used to find all maximal palindromic substrings anywhere within the input string, again in $O(n)$ time. Therefore, it provides an $O(n)$ -time solution to the longest palindromic substring problem. Alternative $O(n)$ -time solutions were provided by Jeuring (1994), and by Gusfield (1997), who described a solution based on suffix trees. A faster algorithm can be achieved in the word RAM model of computation if the size σ of the input alphabet is in $2^{o(\log n)}$. In particular, this algorithm runs in $O(n \log \sigma / \log n)$ time using $O(n \log \sigma / \log n)$ space.^[1] Efficient parallel algorithms are also known for the problem.^[2]

The longest palindromic substring problem should not be confused with the different problem of finding the longest palindromic subsequence.

Slower algorithm

This algorithm is slower than Manacher's algorithm, but is a good stepping stone for understanding Manacher's algorithm. It looks at each character as the center of a palindrome and loops to determine the largest palindrome with that center.

The loop at the center of the function only works for palindromes where the length is an odd number. The function works for even-length palindromes by modifying the input string. The character '|' is inserted between every character in the inputs string, and at both ends. So the input "book" becomes "|b|o|o|k|". The even-length palindrome "oo" in "book" becomes the odd-length palindrome "|o|o|".

```
Longest_Palindrome_SLOW(string S, string S') {
    // S' == S with a bogus character (eg. '|') inserted
    // between each character (including outer boundaries)

    // The radius of the longest palindrome centered on each place in S'
    // note: length(S') = length(PalindromeRadii) = 2 * length(S) + 1

    array PalindromeRadii = [0,...,0]

    Center = 0

    while Center < length(S') {
        // Determine the longest palindrome starting
        // at Center-Radius and going to Center+Radius
        Radius = 0

        while Center-(Radius + 1) >= 0 and
            Center+(Radius + 1) < length(S') and
            S'[Center-(Radius + 1)] = S'[Center+(Radius + 1)] {
            Radius = Radius + 1
        }

        // Save the radius of the longest palindrome in the array

        PalindromeRadii[Center] = Radius

        Center = Center + 1
    }

    // One can show that longest_palindrome_in_S is max(PalindromeRadii).
    // if S'[i] == '|', PalindromeRadii[i] is even, otherwise you could increase PalindromeRadii[i] by 1,
    // which is equivalent to inserting an extra '|' in each border.
    // Remember that a palindrome centered in an '|' in S' corresponds to an even palindrome in S.
    // if S'[i] != '|', PalindromeRadii[i] is odd (same argument), and corresponds to an odd palindrome.
    // In this case, the length of the palindrome
    // centered at that character is also x=PalindromeRadii[i], as we have (x-1)/2 characters on each side,
    // plus the extra middle one ((x-1)/2*2+1=x)
    longest_palindrome_in_S = max(PalindromeRadii)

    return longest_palindrome_in_S
}
```

The runtime of this algorithm is $O(n^2)$. The outer loop runs n times and the inner loop can run up to $n/2$ times.

Manacher's algorithm

Below is the pseudocode for Manacher's algorithm. The algorithm is faster than the previous algorithm because it exploits when a palindrome happens inside another palindrome.

For example, consider the input string "abacaba". By the time it gets to the "c", Manacher's algorithm will have identified the length of every palindrome centered on the letters before the "c". At the "c", it runs a loop to identify the largest palindrome centered on the "c": "abacaba". With that knowledge, everything after the "c" looks like the reflection of everything before the "c". The "a" after the "c" has the same longest palindrome as the "a" before the "c". Similarly, the "b" after the "c" has a longest palindrome that is *at least* the length of the longest palindrome centered on the "b" before the "c". There are some special cases to consider, but that trick speeds up the computation dramatically.

```
Longest_Palindrome(string S, string S') {
    // S' == S with a bogus character (eg. '|') inserted
    // between each character (including outer boundaries)

    // The radius of the longest palindrome centered on each place in S'
    // note: length(S') = length(PalindromeRadii) = 2 × length(S) + 1

    array PalindromeRadii = [0,...,0]

    Center = 0
    Radius = 0

    while Center < length(S') {
        // At the start of the loop, Radius is already set to a lower-bound
        // for the longest radius. In the first iteration, Radius is 0, but
        // it can be higher on later iterations.

        // Determine the longest palindrome starting at Center-Radius and
        // going to Center+Radius

        while Center-(Radius+1) >= 0 and
              Center+(Radius+1) < length(S') and
              S'[Center-(Radius+1)] = S'[Center+(Radius+1)] {
            Radius = Radius+1
        }

        // Save the radius of the longest palindrome in the array
        PalindromeRadii[Center] = Radius

        // Below, Center is incremented.
        // If any precomputed values can be reused, they are.
        // Also, Radius may be set to a value greater than 0

        OldCenter = Center
        OldRadius = Radius
        Center = Center+1

        // Radius' default value will be 0, if we reach the end of the
        // following loop.
        Radius = 0

        while Center <= OldCenter + OldRadius {

            // Because Center lies inside the old palindrome and every
            // character inside a palindrome has a "mirrored" character
            // reflected across its center, we can use the data that was
            // precomputed for the Center's mirrored point.

            MirroredCenter = OldCenter - (Center - OldCenter)
            MaxMirroredRadius = OldCenter + OldRadius - Center

            if PalindromeRadii[MirroredCenter] < MaxMirroredRadius {

                // The palindrome centered at MirroredCenter is entirely
                // contained in the palindrome centered at OldCenter So,
                // MirroredCenter and Center have the same sized palindrome

                PalindromeRadii[Center] = PalindromeRadii[MirroredCenter]
                Center = Center+1
            } else if PalindromeRadii[MirroredCenter] > MaxMirroredRadius {

                // The palindrome at MirroredCenter extends beyond the
                // palindrome at OldCenter The palindrome at Center must
                // end at the edge of the OldCenter palindrome Otherwise,
                // the palindrome at OldCenter would be bigger

                PalindromeRadii[Center] = MaxMirroredRadius
                Center = Center+1
            } else { // PalindromeRadii[MirroredCenter] = MaxMirroredRadius

                // Since the palindrome at MirroredCenter ends exactly at
                // the edge of the palindrome centered at OldCenter, the
                // palindrome at Center might be bigger Set Radius to the
                // minimum size of the palindrome at Center so it doesn't
                // recheck unnecessarily

                Radius = MaxMirroredRadius
            }
        }
    }
}
```

```

        break
    }
}

// A palindrome's size is equal to its radius * 2. However, since our
// variable Radius considers our bogus characters to the side of the
// center, the size of its corresponding palindrome is actually 2 *
// (Radius / 2), which means a palindrome's size is equal to its
// corresponding Radius value in PalindromeRadii

longest_palindrome_in_S = max(PalindromeRadii)
return longest_palindrome_in_S
}

```

Special cases

Manacher's algorithm is faster because it reuses precomputed data when a palindrome exists inside another palindrome. There are 3 cases of this. They are represented by the "if / else if / else" statement in the pseudocode.

The first case is when the palindrome at `MirroredCenter` lies completely inside the "Old" palindrome. In this situation, the palindrome at `Center` will have the same length as the one at `MirroredCenter`. For example, if the "Old" palindrome is "abcbpbcba", we can see that the palindrome centered on "c" after the "p" must have the same length as the palindrome centered on the "c" before the "p".

The second case is when the palindrome at `MirroredCenter` extends outside the "Old" palindrome. That is, it extends "to the left" (or, contains characters with a lower index than any inside the "Old" palindrome). Because the "Old" palindrome is the largest possible palindrome centered on `OldCenter`, we know the characters before and after it are different. Thus, the palindrome at `Center` will run exactly up to the border of the "Old" palindrome, because the next character will be different than the one inside the palindrome at `MirroredCenter`. For example, if the string was "ababc", the "Old" palindrome could be "bab" with the `Center` being the second "b" and the `MirroredCenter` being the first "b". Since the palindrome at the `MirroredCenter` is "aba" and extends beyond the boundaries of the "Old" palindrome, we know the longest palindrome at the second "b" can only extend up to the border of the "Old" palindrome. We know this because if the character after the "Old" palindrome had been an "a" instead of a "c", the "Old" palindrome would have been longer.

The third and last case is when the palindrome at `MirroredCenter` extends exactly up to the border of the "Old" palindrome. In this case, we don't know if the character after the "Old" palindrome might make the palindrome at `Center` longer than the one at `MirroredCenter`. But we do know that the palindrome at `Center` is *at least* as long as the one at `MirroredCenter`. In this case, `Radius` is initialized to the radius of the palindrome at `MirroredCenter` and the search starts from there. An example string would be "abcbpbcbp" where the "Old" palindrome is "bcbpbc" and the `Center` is on the second "c". The `MirroredCenter` is the first "c" and it has a longest palindrome of "bcb". The longest palindrome at the `Center` on the second "c" has to be at least that long and, in this case, is longer.

Runtime

The algorithm runs in linear time. This can be seen by noting that `Center` strictly increases after each outer loop and the sum `Center + Radius` is non-decreasing. Moreover, the number of operations in the first inner loop is linear in the increase of the sum `Center + Radius` while the number of operations in the second inner loop is linear in the increase of `Center`. Since $\text{Center} \leq 2n+1$ and $\text{Radius} \leq n$, the total number of operations in the first and second inner loops is $O(n)$ and the total number of operations in the outer loop, other than those in the inner loops, is also $O(n)$. The overall running time is therefore $O(n)$.

Notes

- Charalampopoulos, Panagiotis; Pissis, Solon P.; Radoszewski, Jakub (Jun 2022). Bannai, Hideo; Holub, Jan (eds.). *Longest Palindromic Substring in Sublinear Time*. Combinatorial Pattern Matching. Leibniz International Proceedings in Informatics (LIPIcs). Vol. 223. Schloss Dagstuhl. doi:10.4230/LIPIcs.CPM.2022.20 (https://doi.org/10.4230%2FLIPIcs.CPM.2022.20). Here: Theorem 1, p.20:2.

2. Crochemore & Rytter (1991), Apostolico, Breslauer & Galil (1995).

See also

- [Palindrome tree](#)

References

- Apostolico, Alberto; Breslauer, Dany; Galil, Zvi (1995), "Parallel detection of all palindromes in a string" (<https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=2068&context=cstech>), *Theoretical Computer Science*, **141** (1–2): 163–173, doi:10.1016/0304-3975(94)00083-U (<https://doi.org/10.1016%2F0304-3975%2894%2900083-U>).
- Charalampopoulos, Panagiotis; Pissis, Solon P.; Radoszewski, Jakub (2022), "Longest Palindromic Substring in Sublinear Time" (<https://doi.org/10.4230/LIPIcs.CPM.2022.20>), *33rd Annual Symposium on Combinatorial Pattern Matching, CPM 2022, June 27–29, 2022, Prague, Czech Republic*, **223**: 20:1–20:9, doi:10.4230/LIPIcs.CPM.2022.20 (<https://doi.org/10.4230%2FLIPIcs.CPM.2022.20>).
- Crochemore, Maxime; Rytter, Wojciech (1991), "Usefulness of the Karp–Miller–Rosenberg algorithm in parallel computations on strings and arrays", *Theoretical Computer Science*, **88** (1): 59–82, doi:10.1016/0304-3975(91)90073-B (<https://doi.org/10.1016%2F0304-3975%2891%2990073-B>), MR 1130372 (<https://mathscinet.ams.org/mathscinet-getitem?mr=1130372>).
- Crochemore, Maxime; Rytter, Wojciech (2003), "8.1 Searching for symmetric words", *Jewels of Stringology: Text Algorithms*, World Scientific, pp. 111–114, ISBN 978-981-02-4897-0.
- Gusfield, Dan (1997), "9.2 Finding all maximal palindromes in linear time", *Algorithms on Strings, Trees, and Sequences*, Cambridge: Cambridge University Press, pp. 197–199, doi:10.1017/CBO9780511574931 (<https://doi.org/10.1017%2FCBO9780511574931>), ISBN 0-521-58519-8, MR 1460730 (<https://mathscinet.ams.org/mathscinet-getitem?mr=1460730>).
- Jeuring, Johan (1994), "The derivation of on-line algorithms, with an application to finding palindromes", *Algorithmica*, **11** (2): 146–184, doi:10.1007/BF01182773 (<https://doi.org/10.1007%2FBF01182773>), hdl:1874/20926 (<https://hdl.handle.net/1874%2F20926>), MR 1272521 (<https://mathscinet.ams.org/mathscinet-getitem?mr=1272521>), S2CID 7032332 (<https://api.semanticscholar.org/CorpusID:7032332>).
- Manacher, Glenn (1975), "A new linear-time "on-line" algorithm for finding the smallest initial palindrome of a string", *Journal of the ACM*, **22** (3): 346–351, doi:10.1145/321892.321896 (<https://doi.org/10.1145%2F321892.321896>), S2CID 10615419 (<https://api.semanticscholar.org/CorpusID:10615419>).
-  This article incorporates Longest palindromic substring text (https://web.archive.org/web/20180629091030/http://wcipeg.com/wiki/index.php?title=Longest_palindromic_substring) available under the CC BY 3.0 license.

External links

- *Longest Palindromic Substring Part II*. (<https://web.archive.org/web/20181208031518/https://articles.leetcode.com/longest-palindromic-substring-part-ii/>), 2011-11-20, archived from the original (<https://articles.leetcode.com/longest-palindromic-substring-part-ii/>) on 2018-12-08. A description of Manacher's algorithm for finding the longest palindromic substring in linear time.
- Akalin, Fred (2007-11-28), *Finding the longest palindromic substring in linear time* (<https://www.akalin.com/longest-palindrome-linear-time>), retrieved 2016-10-01. An explanation and Python implementation of Manacher's linear-time algorithm.
- Jeuring, Johan (2007–2010), *Palindromes* (<http://www.staff.science.uu.nl/~jeuri101/homepage/palindromes/index.html>), retrieved 2011-11-22. Haskell implementation of Jeuring's linear-time algorithm.
- *Palindromes* (<https://github.com/vvikas/palindromes/blob/master/src/LongestPalindromicSubString.java>) (deadlink). Java implementation of Manacher's linear-time algorithm.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Longest_palindromic_substring&oldid=1213016817"

▪